

İTÜ Rover Takımı

Git ve Github Dokümantasyonu



Yiğit Cılboğlu

28 Ekim 2025

İçindekiler

1	Git	3
2	Git Komutları ve Kavramları	3
2.1	Repository	3
2.2	Temel Komutlar	3
2.3	Hash	4
2.4	Lokasyonlar	4
2.5	Branch	5
2.6	Merge	5
2.7	Rebase	6
2.8	Geri Alma	7
2.9	Remote	7
2.10	Fetch	7
3	GitHub	7
3.1	Push	7
3.2	Pull	8
3.3	Pull Requests	8
4	Takım İçi İş Akışı	8
4.1	Software-26 İçin	9
4.2	Diğer Repository'ler İçin	9
5	Bazı Git Özellikleri	9
5.1	.gitignore	9
5.2	Fork	10
5.3	reflog	10
5.4	Merge Conflict	10
5.5	Squash	10
5.6	Stash	10
5.7	Revert	11
5.8	Tag	11

1 Git

Git bir versiyon kontrol sistemidir. Dosyalarınızın değişikliklerini yerel bilgisayarınızda takip etmenizi sağlar. Zamanında proje yönetimi maksadıyla BitKeeper kullanan Linus Torvalds bu uygulamanın performans sorunlarından ötürü hiç memnun kalmamıştı. Bu yüzden de oturup 6 gün içerisinde Git'in ilk versiyonunu yazmıştı.

Git içerisinde dosya değişiklikleriniz "commit"ler altında kaydedilir; kimin, ne zaman, hangi değişiklikleri yaptığı bilinir ve hata ayıklamak, projeyi yönetmek sürdürülebilir bir hale getirilir. Git, GitHub'dan bağımsızdır. Git açık kaynaklı bir yazılımdır, GitHub'ın sahibi ise Microsoft'tur ve ayrıca GitHub'ın alternatifi olan GitLab, Azure DevOps gibi yazılımlar da bulunur.

2 Git Komutları ve Kavramları

2.1 Repository

Git içerisinde veriler repository'ler aracılığı ile yönetilir. Repository, bir projenin tüm dosyalarını, değişiklik geçmişini ve commit bilgilerini saklayan yerdir. Her repository, proje üzerinde yapılan değişiklikleri takip etmek için commitler kullanır ve bu sayede kodun geçmişi eksiksiz bir şekilde korunur.

2.2 Temel Komutlar

Git kullanarak bir repository başlatılmak istendiğinde, repository olması için seçilen klasörün içerisinde şu komut kullanılır:

```
git init
```

Git repository'nizin GitHub ile bir bağlantısı olması gerekmemektedir. Repository'nizin durumu hakkında bilgi almak için şu komutu girebilirsiniz:

```
git status
```

Repository'niz içerisinde bir değişiklik yaptıktan sonra bu değişikliği commit etmek istersiniz. Bunun için öncelikle yaptığınız değişiklikleri sahneye almanız gerekmektedir. Bunu şu komutla yapabilirsiniz:

```
git add .
```

Bu sahneye aldığınız değişiklikleri tarihe ve projeye kalıcı olarak kaydetmek için "git commit" komutunu kullanırsınız. Commit işlemi, değişiklikleri repository'nin geçmişine ekler ve üzerinde çalıştığınız proje üzerinde yapılan değişikliklerin takip edilmesini sağlar. Örneğin:

```
git commit -m "Commit Mesajınızı buraya yazın."
```

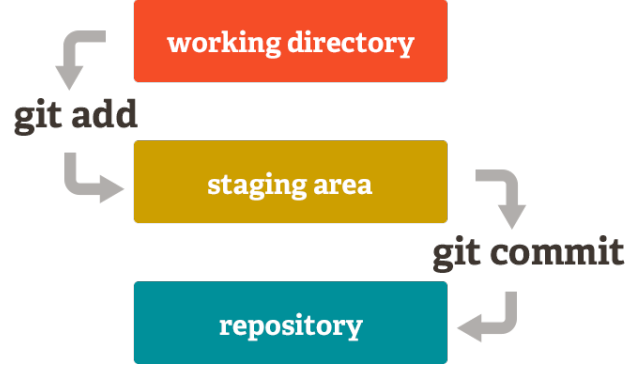
Commit komutundaki -m flag'ı bu komutu direk komut satırından verebilmeniz içindir. Bu flag'ı kullanmazsanız bir text editor açılır.

Bir çok değişiklik ve commit'ten sonra artık projenizin bir tarihi olur. Repository'nin geçmişini görmek için şu komutu girebilirsiniz:

```
git log
```

Aynı komutun çıktısı daha açıklayıcı ve anlaşılır olsun isterseniz, şu şekilde de kullanabilirsiniz:

```
git log --oneline --graph --decorate --all
```



Figür 1: Git Staging Aşamaları

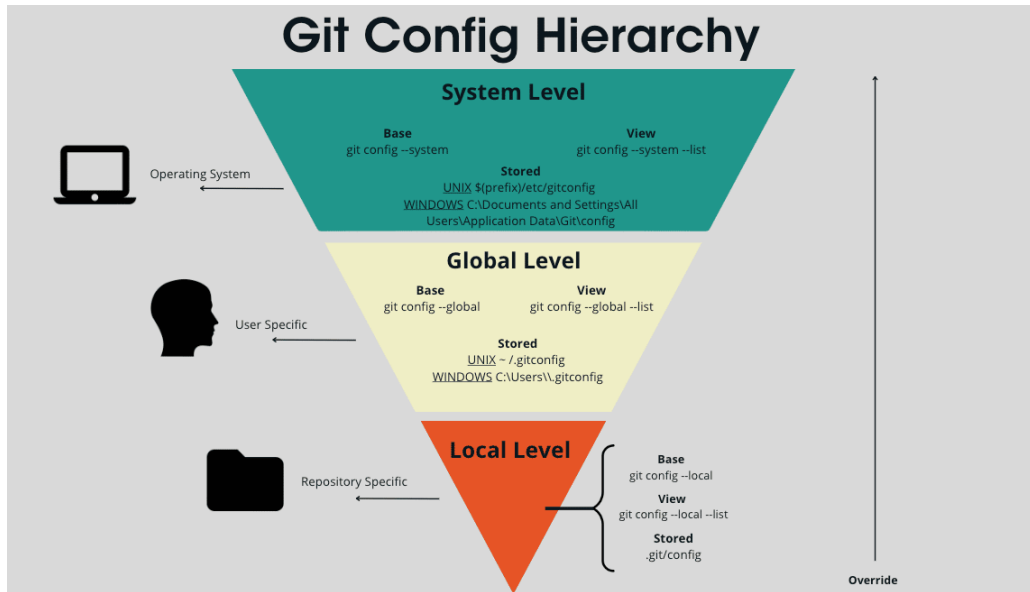
2.3 Hash

Git'te hash, her commit'in benzersiz kimliğini temsil eden bir değerdir. Her bir commit için farklıdır ve benzersizdir. Hash'ler belirli commit'lere referans etmek için kullanılır. örneğin:

```
git cat-file -p <commit_hash>
```

Bu komut sayesinde referans edilen commit'in içeriğini görüntüleyebilirsiniz.

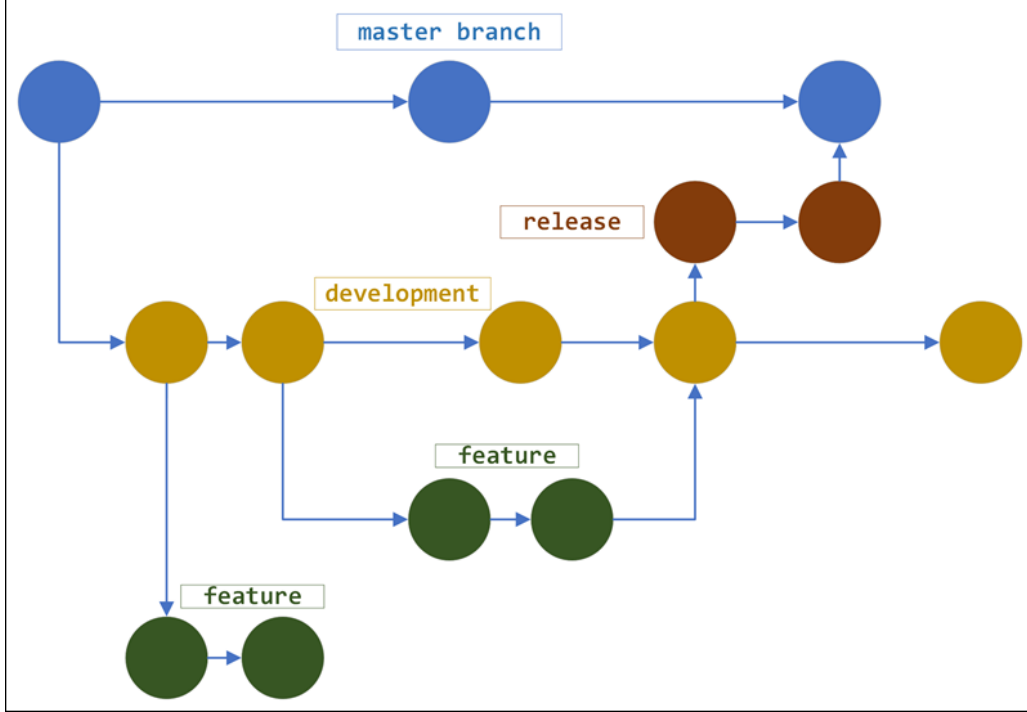
2.4 Lokasyonlar



Figür 2: Git Hiyerarşisi

2.5 Branch

Branch'ler belki de Git hakkında anlaşılması en çok önem arz eden konseptlerden biridir. Git'te branch, bir projenin bağımsız olarak geliştirilebilecek paralel sürümünü temsil eder. Branch'ler, aynı repository içinde birden fazla geliştirme hattını yönetmenizi sağlar.



Figür 3: Git Branch Şeması

Bir repository'de hangi branch'te bulunduğunuzu görmek için şu komutu kullanabilirsiniz:

```
git branch
```

Bir branch'in adını değiştirmek için şu komut kullanılabilir:

```
git branch -m <old_name> <new_name>
```

Yeni bir branch yaratıp ardından da o branch'e geçmek için şu komut kullanılır:

```
git switch -c <branch_name>
```

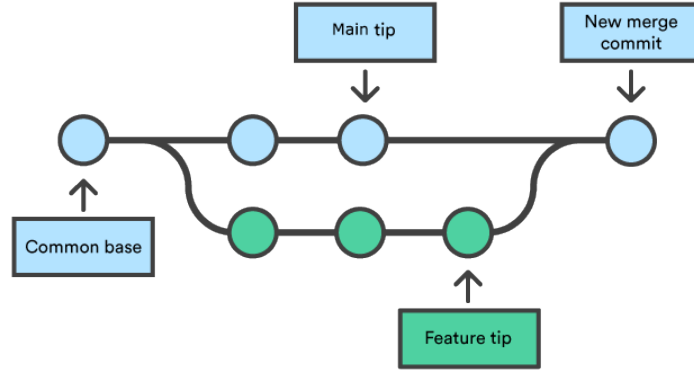
Buradaki -c flag'ı yeni bir branch oluşturulacağını gösterir. Yalnızca branch'ler arasında geçiş yapmak için şu komut kullanılabilir:

```
git switch <branch_name>
```

2.6 Merge

Git'te merging, farklı branch'lerde yapılan değişiklikleri birleştirme işlemidir. Genellikle bir özellik branch'i üzerinde geliştirme yapıldıktan sonra bu branch'in ana branch ile birleştirilmesi gerekir. Merge işlemi, iki branch'in commit geçmişlerini bir araya getirir ve değişiklikleri tek bir branch üzerinde toplar. Git, değişiklikler farklı dosya veya satırlarda yapıldıysa otomatik olarak birleştirme yapabilir; ancak aynı satır üzerinde çelişkili değişiklikler varsa merge conflict oluşur ve bu durumda kullanıcı müdahalesi gerekir. Merge tamamlandığında,

branch'ler kendi geçmişlerini korur ve tüm commit'ler birleşik bir geçmişte görünür. Böylece paralel geliştirme yapılabilmiş ve proje tutarlı bir şekilde güncel hâle getirilmiş olur.



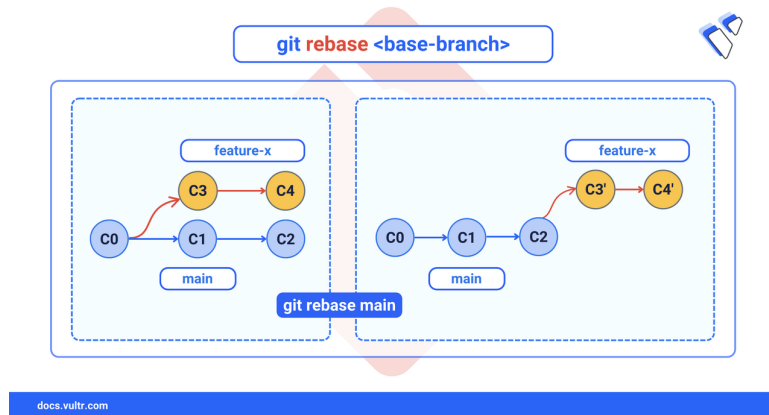
Figür 4: Merge işlemi

Merge yapmak için şu komut kullanılabilir:

```
git merge <branch_name>
```

2.7 Rebase

Git'te rebase, bir branch'in tabanını başka bir branch'in son commit'ine taşıyarak commit geçmişini daha düz ve temiz bir çizgi hâline getirme işlemidir. Rebase genellikle bir feature branch'i ana branch'in güncel commit'leriyle senkronize etmek için kullanılır. Rebase yapıldığında, feature branch'teki commit'ler geçici olarak alınır, ana branch'in son commit'inin üzerine tekrar uygulanır. Bu sayede commit geçmişi, merge commit'leri olmadan lineer bir şekilde sıralanır, yani paralel gelişmeler tek bir düz çizgi gibi görünür. Ancak dikkat edilmelidir ki, rebase yapmak tarihi değiştirir ve sakıncalı olabilir.



Figür 5: Rebase işlemi

Rebase yapmak için şu komut kullanılabilir:

```
git rebase main
```

2.8 Geri Alma

Git'te reset, commit geçmişini ve çalışma alanını belirli bir commit'e geri almak için kullanılır. Bunun iki temel türü vardır: soft reset ve hard reset, ve aralarındaki fark işlevleri açısından önemlidir.

- Soft reset commit işaretçisini belirtilen commit'e taşır, ancak staging area (index) ve çalışma dizini değişmez. Yani commit'ler geri alınmış gibi görünür, ama değişiklikler hâlâ sahnelenmiş durumda kalır; bu sayede değişiklikleri yeniden commit edebilir veya düzenleyebilirsiniz.
- Hard reset commit işaretçisini taşırken hem staging area'yı hem de çalışma dizinini belirtilen commit'in durumuna getirir. Bu, commit sonrası yapılan tüm değişikliklerin tamamen silinmesi anlamına gelir; dikkatli kullanılmalıdır, çünkü geri alınamaz.

Git reset için komutlar:

```
git reset --soft <commit_hash>
git reset --hard <commit_hash>
```

2.9 Remote

Git'te remote, yerel repository'nizin uzaktaki bir kopyasına işaret eden bir referanstır. Genellikle GitHub, GitLab, Bitbucket gibi sunucularda barındırılan repository'ler remote olarak eklenir. Remote sayesinde, yerel değişikliklerinizi uzaktaki repository'ye gönderebilir (git push), uzak repository'deki değişiklikleri alabilir (git pull) veya sadece inceleyebilirsiniz (git fetch). Remote'lar, birden fazla kişiyle işbirliği yapmayı ve projeyi senkronize tutmayı sağlar; her remote, bir isim (ör. origin) ve bir URL ile tanımlanır. Bu yapı, yerel ve uzak repository'ler arasında veri akışını yöneterek takım çalışmalarını düzenler ve versiyon kontrolünü merkezileştirir.

2.10 Fetch

Git'te fetch, uzak repository'deki (remote) değişiklikleri yerel repository'nize indirip güncellemek için kullanılan bir komuttur, ancak bu değişiklikleri otomatik olarak çalışma dizininize veya aktif branch'inize uygulamaz. Yani fetch, uzak branch'lerdeki yeni commit'leri, tag'leri veya dalları yerel kopyanızda güncel olarak tutar, ama mevcut branch'inizi etkilemez; böylece önce değişiklikleri inceleme fırsatı sunar.

3 GitHub

Bu başlık altında inceleyeceğimiz özellikler Git'ten ziyade GitHub ile alakalıdır. Unutulmamalıdır ki Git ve GitHub iki farklı şeydir.

3.1 Push

Uzak repository'e (remote) değişikliklerinizi göndermek için şu komutu kullanabilirsiniz:

```
git push origin main
```

3.2 Pull

Eskaza bir merge conflict'e yahut bir soruna sebep olmamak amacı ile bir repository üzerine çalışmaya başlamadan hep o yerel repository ile uzak repository'nin(remote) aynı olduğuna emin olmalıyız. Bunun için şu komut kullanılabilir:

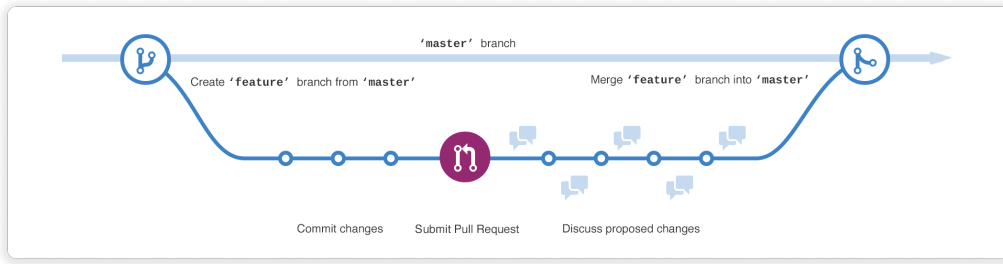
```
git pull <remote> <branch>
```

Ama çoğunlukla:

```
git pull origin main
```

3.3 Pull Requests

Pull request, Git tabanlı platformlarda (GitHub, GitLab vb.) bir branch'te yaptığımız değişikliklerin başka bir branch'e birleştirilmesi için talep olarak gönderilmesidir. Pull request, kodun gözden geçirilmesini, tartışılmasını ve onaylanmasını sağlar. Takım çalışmasında, geliştiriciler değişikliklerini doğrudan ana branch'e pushlamak yerine pull request oluşturur; böylece kod incelemesi yapılır, hata veya uyumsuzluklar tespit edilir ve onaylandıktan sonra merge işlemi gerçekleştirilir. Bu mekanizma, işbirliğini ve kod kalitesini artırmak için kritik bir süreçtir.



Figür 6: Pull Request İşlemi

4 Takım İçi İş Akışı

İtü Rover Takımı Yazılım Alt Ekibi olarak bizim repository'lerimiz ikiye ayrılmaktadır.

- Software-26
- Diğer repository'ler

Software-26 üzerine direkt bir biçimde değişiklik yapma yetkisi kimsede bulunmamaktadır. Değişiklikler bir branch'e push'landıktan sonra pull request oluşturulur. Bu değişim değerlendirilir ve ancak ardından bu değişiklik ana branch'e eklenir. Takımın tüm paketlerinin ve yazılımlarının toplandığı bir mega repository görevi görür.

Kalan bütün repository'lere pull request gerekmeksizin direkt bir biçimde push yapılabilir. Bu repository'ler genellikle spesifik bir sistemi taşırlar.

4.1 Software-26 İçin

Değişiklik yapmaya başlamadan önce remote ile yerel repository'nizi eşleyin.

```
git pull
```

Değişiklik yapmadan önce yeni bir branch'e geçtiğinizden emin olun.

```
git switch -c <branch_name>
```

Ardından değişikliklerinizi yapın. Bu değişikliklerin ardından commit atın. Bir push bir commit barındırmak zorunda değildir, yerel repository'nizde 3-4 commit atmış olmanız sonradan push atarken sıkıntı çıkarmaz.

```
git add .  
git commit -m "Commit Mesajı"
```

Bu değişikliklerin ardından remote'ta açacağımız bir branch'e push atın.

```
git push -u origin <branch_name>
```

Daha sonra GitHub'a girip pull request oluşturabilirsiniz. Yerel repository'nizde kalmış olan branch sizi rahatsız ederse şu komutları kullanarak silebilirsiniz.

```
git switch main  
git branch -D <branch_name>
```

4.2 Diğer Repository'ler İçin

Değişiklik yapmaya başlamadan önce remote ile yerel repository'nizi eşleyin.

```
git pull
```

Değişikliklerinizi yaptıktan sonra commit atın.

```
git add .  
git commit -m "Commit Mesajı"
```

Remote repository'e push'layın.

```
git push origin main
```

5 Bazı Git Özellikleri

5.1 .gitignore

.gitignore, Git'in hangi dosya ve klasörleri takip etmeyeceğini belirten özel bir dosyadır. Genellikle derleme dosyaları, geçici dosyalar, loglar veya kişisel ayar dosyaları .gitignore içine eklenir, böylece bu dosyalar repository'ye eklenmez ve commit'lenmez. Bu sayede gereksiz veya hassas dosyalar projede paylaşılmaz ve versiyon kontrolü daha düzenli olur.

5.2 Fork

Git'te "fork", genellikle GitHub, GitLab veya Bitbucket gibi barındırma servislerinde, bir depoda (repository) değişiklik yapma izniniz olmadığında veya ana projeyi etkilemeden kendi başınıza denemeler yapmak istediğinizde kullanılan bir işlemdir. Amaç bir projenin kaynak kodunun tamamının sizin hesabınızda ve kontrolünüzde olan bir kopyasını oluşturmaktır. Bu kopya, orijinal depodan bağımsız olarak var olur.

5.3 reflog

HEAD referansının takibini yapan bir komuttur. Şu şekilde kullanılabilir:

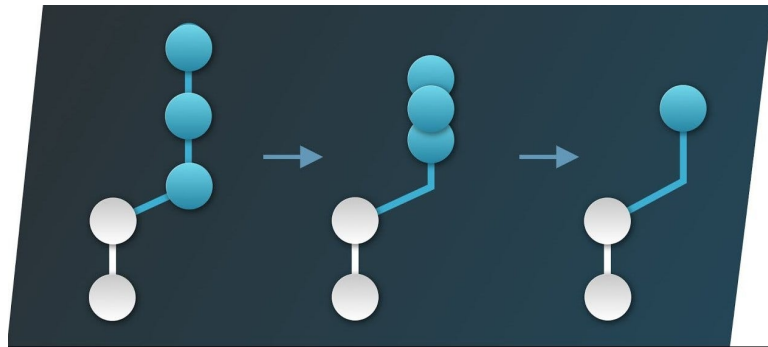
```
git reflog
```

5.4 Merge Conflict

Git'te bir merge conflict, iki farklı branch'i merge'lemeye çalıştığınızda, Git'in aynı dosyanın aynı satırlarında veya birbirine çok yakın kısımlarında yapılmış olan farklı değişiklikleri otomatik olarak nasıl birleştireceğine karar verememesi durumunda ortaya çıkar. Git, hangi değişikliğin korunması gerektiğini bilemediği için birleştirme işlemini durdurur ve bu çakışan alanları özel işaretleyicilerle (|||||, =====, ~~~~~) göstererek, sorunun manuel olarak çözülmesini ister. Çakışmayı çözmek, geliştiricinin her iki daldan gelen kodu incelemesi ve doğru, nihai kodu seçerek dosyayı kaydetmesi ve ardından birleştirme işlemini tamamlaması anlamına gelir.

5.5 Squash

Git squash, bir dizi ardışık commit'i alıp, bu commit'lerin tüm değişikliklerini tek bir yeni ve anlamlı commit halinde birleştiren bir Git işlemidir. Temel amaç, bir özellik geliştirme sürecinde yapılan "küçük düzeltmeler," "yazım hatalarını giderme" veya "devam eden çalışma" gibi gürültü yaratan ve önemsiz commit'leri ana tarihçeye aktarmadan önce temiz ve anlamlı bir kayıt bırakmaktır. Bu sayede, projenin geçmişi daha okunabilir, anlaşılır ve bakımı kolay hale gelir, böylece ekip üyeleri büyük bir özelliği tek bir mantıksal adımla takip edebilirler.



Figür 7: Squash işlemi

5.6 Stash

git stash komutu, üzerinde çalıştığınız ancak henüz commit etmeye hazır olmadığınız (yani staging area'da ya da çalışma dizininde bulunan) geçici değişiklikleri bir kenara, bir stash'e kaydetmenizi sağlayan bir araçtır.

Bu, acil bir hata düzeltmesi veya branch değiştirmek gibi başka bir işe aniden geçmeniz gerektiğinde, mevcut dağınık çalışmanızı kaybetmeden veya anlamsız bir commit oluşturmadan çalışma dizininizi temizlemek için idealdir.

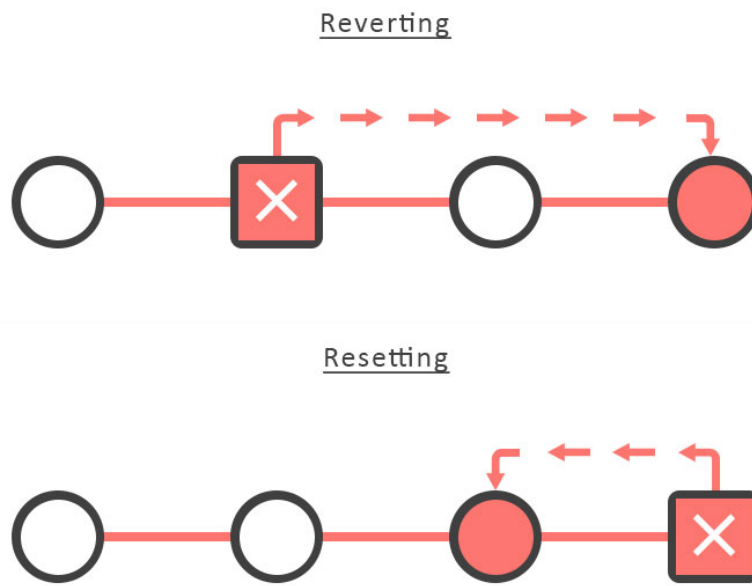
Ardından bu stash'teki değişikliği branch'e uygulamak istediğinizde şu komutu kullanabilirsiniz:

```
git stash pop
```

5.7 Revert

`git revert` komutu, Git tarihçesinden bir commit'i kaldırmanın aksine, belirli bir önceki commit'te yapılan tüm değişiklikleri geri alan yeni bir commit oluşturur. Şu şekilde kullanılır:

```
git revert <commit_hash>
```



Figür 8: Revert ve Reset işlemi

5.8 Tag

Bir commit'e işaret eden bir pointer yaratır. Şu şekilde kullanılabilir:

```
git tag -a "name" -m "message"
```